

10. Currying

Kya hai?

Function ko aise badalna ki `f(a, b, c)` ko `f(a)(b)(c)` ki tarah bhi call kar sako. Currying function ko **call nahi karta, sirf transform karta hai**.

Simple example (2 arguments)

```
js

function curry(f) {
  return function(a) {
    return function(b) {
      return f(a, b);
    };
  };
}

function sum(a, b) { return a + b; }
let curriedSum = curry(sum);
curriedSum(1)(2); // 3
```

Har wrapper ek argument Lexical Environment me yaad rakhta hai.

Lodash ka `_.curry` dono tarike allow karta hai:

```
js

let curriedSum = _.curry(sum);
curriedSum(1, 2); // 3
curriedSum(1)(2); // 3
```

Kyun useful? (Partial functions)

```
js
```

```
function log(date, importance, message) {
  console.log(`[${date.getHours()}:${date.getMinutes()}] [${importance}] ${message}`);
}
log = _.curry(log);

let logNow = log(new Date()); // pehla argument fix
logNow("INFO", "message");

let debugNow = logNow("DEBUG"); // do arguments fix
debugNow("message");
```

`logNow` aur `debugNow` ko **partial function** kehte hain. Original `log` bhi normal tarike se chalta rehta hai.

Advanced curry (kisi bhi argument count ke liye)

```
js

function curry(func) {
  return function curried(...args) {
    if (args.length >= func.length) {
      return func.apply(this, args);
    } else {
      return function(...args2) {
        return curried.apply(this, args.concat(args2));
      };
    }
  };
};

let c = curry((a, b, c) => a + b + c);
c(1, 2, 3); // 6
c(1)(2, 3); // 6
c(1)(2)(3); // 6
```

Logic:

1. Arguments kaafi hain (`func.length` ke barabar ya zyada) to original function chalao.
2. Kam hain to naya wrapper return karo jo purane + naye arguments jodkar phir try kare.

Dhyan rakho

- Function me **fixed number of arguments** hone chahiye. `f(...args)` (rest parameters) wale ko is tarah curry nahi kar sakte.

- Zyaadatar JS implementations normal call bhi allow karte hain, jo strictly "currying" se thoda zyada hai.

Summary

$f(a, b, c) \rightarrow f(a)(b)(c)$. Isse aasani se partial functions ban jaate hain.